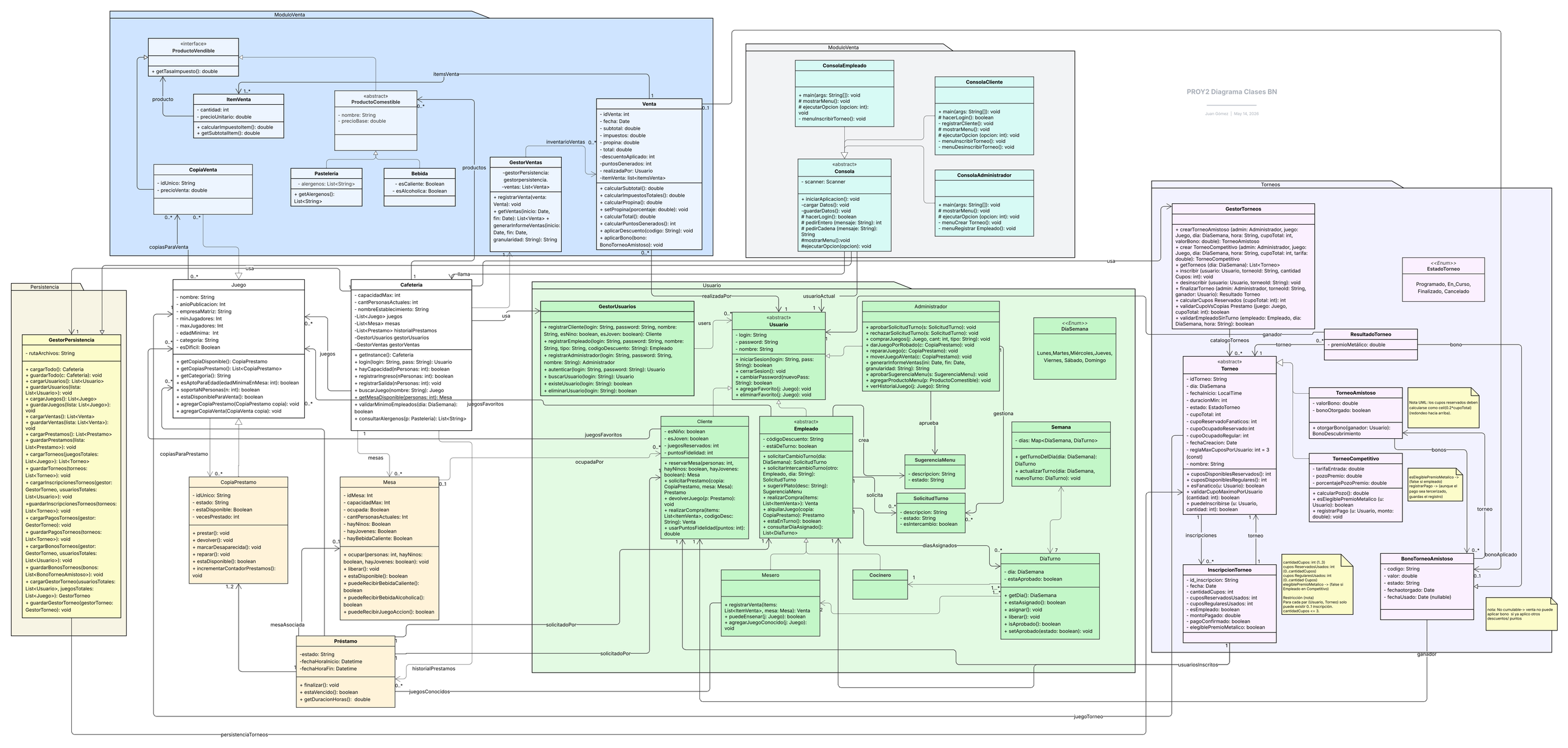
ModuloVenta
«interface»
ProductoVendible
+ getTasaImpuesto(): double
itemsVenta
producto
1..*
ItemVenta
- cantidad: int
- precioUnitario: double
+ calcularImpuestoItem(): double
+ getSubtotalItem(): double
«abstract»
ProductoComestible
- nombre: String
- precioBase: double
0..*
1
Venta
- idVenta: int
- fecha: Date
- subtotal: double
- impuestos: double
- propina: double
- total: double
-descuentoAplicado: int
-puntosGenerados: int
- realizadaPor: Usuario
-ItemVenta: list<ItemsVenta>
+ calcularSubtotal(): double
+ calcularImpuestosTotales(): double
+ calcularPropina(): double
+ setPropina(porcentaje: double): void
+ calcularTotal(): double
+ calcularPuntosGenerados(): int
+ aplicarDescuento(codigo: String): void
+ aplicarBono(bono: BonoTorneoAmistoso): void
0..1
0..*
inventarioVentas
0..*
productos
CopiaVenta
- idUnico: String
- precioVenta: double
0..*
0..*
copiasParaVenta
Pasteleria
- alergenos: List<String>
+ getAlergenos(): List<String>
Bebida
- esCaliente: Boolean
- esAlcoholica: Boolean
GestorVentas
-gestorPersistencia: gestorpersistencia.
-ventas: List<Venta>
+ registrarVenta(venta: Venta): void
+ getVentas(inicio: Date, fin: Date): List<Venta> +
generarInformeVentas(inicio: Date, fin: Date,
granularidad: String): String
1..*
ModuloVenta
ConsolaEmpleado
+ main(args: String[]): void
# mostrarMenu(): void
# ejecutarOpcion (opcion: int): void
- menuInscribirTorneo(): void
ConsolaCliente
+ main(args: String[]): void
# mostrarMenu(): void
# ejecutarOpcion (opcion: int): void
- menuInscribirTorneo(): void
- menuDesinscribirTorneo(): void
«abstract»
Consola
- scanner: Scanner
+ iniciarAplicacion(): void
-cargar Datos(): void
-guardarDatos(): void
hacerLogin(): boolean
pedirEntero (mensaje: String): int
pedirCadena (mensaje: String): String
#mostrarMenu():void
#ejecutarOpcion(opcion): void
ConsolaAdministrador
+ main(args: String[]): void
# mostrarMenu(): void
# ejecutarOpcion (opcion: int): void
- menuCrear Torneo(): void
- menuRegistrar Empleado(): void
PROY2 Diagrama Clases BN
Juan Gómez | May 14, 2026
Persistencia
GestorPersistencia
- rutaArchivos: String
+ cargarTodo(): Cafeteria
+ guardarTodo(c: Cafeteria): void
+ cargarUsuarios(): List<Usuario>
+ guardarUsuarios(lista: List<Usuario>): void
+ cargarJuegos(): List<Juego>
+ guardarJuegos(lista: List<Juego>): void
+ cargarVentas(): List<Venta>
+ guardarVentas(lista: List<Venta>): void
+ cargarPrestamos(): List<Prestamo>
+ guardarPrestamos(lista: List<Prestamo>): void
+ cargarTorneos(juegosTotales: List<Juego>): List<Torneo>
+ guardarTorneos(torneos: List<Torneo>): void
+ cargarInscripcionesTorneos(gestor: GestorTorneo, usuariosTotales: List<Usuario>): void
+guardarInscripcionesTorneos(torneos: List<Torneo>): void
+ cargarPagosTorneos(gestor: GestorTorneo): void
+ guardarPagosTorneos(torneos: List<Torneo>): void
+ cargarBonosTorneos(gestor: GestorTorneo, usuariosTotales: List<Usuario>): void
+ guardarBonosTorneos(bonos: List<BonoTorneoAmistoso>): void
+ cargarGestorTorneo(usuariosTotales: List<Usuario>, juegosTotales: List<Juego>): GestorTorneo
+ guardarGestorTorneo(gestorTorneo: GestorTorneo): void
usa
llama
Juego
- nombre: String
- anioPublicacion: Int
- empresaMatriz: String
- minJugadores: Int
- maxJugadores: Int
- edadMinima: Int
- categoria: String
- esDificil: Boolean
+ getCopiaDisponible(): CopiaPrestamo
+ getCopiasPrestamo(): List<CopiaPrestamo>
+ getCategoria(): String
+ esAptoParaEdad(edadMinimaEnMesa: int): boolean
+ soportaNPersonas(n: int): boolean
+ estaDisponibleParaVenta(): boolean
+ agregarCopiaPrestamo(CopiaPrestamo copia): void
+ agregarCopiaVenta(CopiaVenta copia): void
0..*
0..*
juegos
Cafeteria
- capacidadMax: int
- cantPersonasActuales: int
- nombreEstablecimiento: String
-List<Juego> juegos
-List<Mesa> mesas
-List<Prestamo> historialPrestamos
-GestorUsuarios gestorUsuarios
-GestorVentas gestorVentas
+ getInstance(): Cafeteria
+ login(login: String, pass: String): Usuario
+ hayCapacidad(nPersonas: int): boolean
+ registrarIngreso(nPersonas: int): boolean
+ registrarSalida(nPersonas: int): void
+ buscarJuego(nombre: String): Juego
+ getMesaDisponible(personas: int): Mesa
+ validarMinimoEmpleados(dia: DiaSemana): boolean
+ consultarAlergenos(p: Pasteleria): List<String>
1
0..*
juegosFavoritos
1
mesas
0..*
Usuario
GestorUsuarios
+ registrarCliente(login: String, password: String, nombre: String, esNino: boolean, esJoven: boolean): Cliente
+ registrarEmpleado(login: String, password: String, nombre: String, tipo: String, codigoDescuento: String): Empleado
+ registrarAdministrador(login: String, password: String, nombre: String): Administrador
+ autenticar(login: String, password: String): Usuario
+ buscarUsuario(login: String): Usuario
+ existeUsuario(login: String): boolean
+ eliminarUsuario(login: String): boolean
realizadaPor
0..*
users
«abstract»
Usuario
- login: String
- password: String
- nombre: String
+ iniciarSesion(login: String, pass: String): boolean
+ cerrarSesion(): void
+ cambiarPassword(nuevoPass: String): boolean
+ agregarFavorito(j: Juego): void
+ eliminarFavorito(j: Juego): void
usuarioActual
1
1
Administrador
+ aprobarSolicitudTurno(s: SolicitudTurno): void
+ rechazarSolicitudTurno(s: SolicitudTurno): void
+ compraraJuegos(j: Juego, cant: int, tipo: String): void
+ darJuegoPorRobado(c: CopiaPrestamo): void
+ reparar Juego(c: CopiaPrestamo): void
+ moverJuegoAVenta(c: CopiaPrestamo): void
+ generarInformeVentas(ini: Date, fin: Date, granularidad: String): String
+ aprobarSugerenciaMenu(s: SugerenciaMenu): void
+ agregarProductoMenu(p: ProductoComestible): void
+ verHistorialJuego(j: Juego): String
<<Enum>>
DiaSemana
Lunes,Martes,Miércoles,Jueves, Viernes, Sábado, Domingo
Cliente
- esNiño: boolean
- esJoven: boolean
- juegosReservados: int
- puntosFidelidad: int
+ reservarMesa(personas: int, hayNinos: boolean, hayJovenes: boolean): Mesa
+ solicitarPrestamo(copia: CopiaPrestamo, mesa: Mesa): Prestamo
+ devolverJuego(p: Prestamo): void
+ realizarCompra(items: List<ItemVenta>, codigoDesc: String): Venta
+ consultarDiaAsignado(): List<DiaTurno>
juegosFavoritos
0..*
ocupadaPor
1
«abstract»
Empleado
- códigoDescuento: String
- estáDeTurno: boolean
+ solicitarCambioTurno(dia: DiaSemana): SolicitudTurno
+ solicitarIntercambioTurno(otro: Empleado, dia: String): SolicitudTurno
+ sugerirPlato(desc: String): SugerenciaMenu
+ realizarCompra(items: List<ItemVenta>): Venta
+ alquilarJuego(copia: CopiaPrestamo): Prestamo
+ estaEnTurno(): boolean
+ usarPuntosFidelidad(puntos: int): double
crea
1
1
aprueba
SugerenciaMenu
- descripcion: String
- estado: String
solicita
SolicitudTurno
- descripcion: String
- estado: String
- esIntercambio: boolean
0..*
0..*
Semana
- días: Map<DiaSemana, DiaTurno>
+ getTurnoDelDia(dia: DiaSemana): DiaTurno
+ actualizarTurno(dia: DiaSemana, nuevoTurno: DiaTurno): void
gestiona
1
1
diasAsignados
0..*
7
DiaTurno
- dia: DiaSemana
- estaAprobado: boolean
+ getDia(): DiaSemana
+ estaAsignado(): boolean
+ asignar(): void
+ liberar(): void
+ isAprobado(): boolean
+ setAprobado(estado: boolean): void
1..*
1..*
Mesero
+ registrarVenta(items: List<ItemVenta>, mesa: Mesa): Venta
+ puedeEnsenar(j: Juego): boolean
+ agregarJuegoConocido(j: Juego): void
2
Cocinero
1
CopiaPrestamo
- idUnico: String
- estado: String
- estaDisponible: Boolean
- vecesPrestado: int
+ prestar(): void
+ devolver(): void
+ marcarDesaparecida(): void
+ reparar(): void
+ estaDisponible(): boolean
+ incrementarContadorPrestamos(): void
1..2
Mesa
- idMesa: Int
- capacidadMax: Int
- ocupada: Boolean
- cantPersonasActuales: Int
- hayNinos: Boolean
- hayJovenes: Boolean
- hayBebidaCaliente: Boolean
+ ocupar(personas: int, hayNinos: boolean, hayJovenes: boolean): void
+ liberar(): void
+ estaDisponible(): boolean
+ puedeRecibirBebidaCaliente(): boolean
+ puedeRecibirBebidaAlcoholica(): boolean
+ puedeRecibirJuegoAccion(): boolean
0..1
0..3
mesaAsociada
Préstamo
-estado: String
-fechaHoraInicio: Datetime
-fechaHoraFin: Datetime
+ finalizar(): void
+ estaVencido(): boolean
+ getDuracionHoras(): double
1
1
1
0..*
solicitadoPor
solicitadoPor
historialPrestamos
juegosConocidos
persistenciaTorneos
juegoTorneo
usa
Torneos
GestorTorneos
+ crearTorneoAmistoso (admin: Administrador, juego: Juego, dia: DiaSemana, hora: String, cupoTotal: int, valorBono: double): TorneoAmistoso
+ crear TorneoCompetitivo (admin: Administrador, juego: Juego, dia: DiaSemana, hora: String, cupoTotal: int, tarifa: double): TorneoCompetitivo
+ getTorneos (dia: DiaSemana): List<Torneo>
+ inscribir (usuario: Usuario, torneoId: String, cantidad Cupos: int): void
+ desinscribir (usuario: Usuario, torneoId: String): void
+ finalizarTorneo (admin: Administrador, torneoId: String, ganador: Usuario): Resultado Torneo
+ calcularCupos Reservados (cupoTotal: int): int
+ validarCupoVsCopias Prestamo (juego: Juego, cupoTotal: int): boolean
+ validarEmpleadoSinTurno (empleado: Empleado, dia: DiaSemana, hora: String): boolean
<<Enum>>
EstadoTorneo
Programado, En_Curso, Finalizado, Cancelado
catalogoTorneos
0..*
1
ganador
torneo
0..*
ResultadoTorneo
- premioMetálico: double
bono
«abstract»
Torneo
- idTorneo: String
- dia: DiaSemana
- fechaInicio: LocalTime
- duracionMin: int
- estado: EstadoTorneo
- cupoTotal: int
- cupoReservadoFanaticos: int
- cupoOcupadoReservado:int
- cupoOcupadoRegular: int
- fechaCreacion: Date
- reglaMaxCuposPorUsuario: int = 3 (const)
- nombre: String
+ cuposDisponiblesReservados(): int
+ cuposDisponiblesRegulares(): int
+ esFanatico(u: Usuario): boolean
+ validarCupoMaximoPorUsuario (cantidad: int): boolean
+ puedeInscribirse (u: Usuario, cantidad: int): boolean
1
1
inscripciones
torneo
0..*
1
usuariosInscritos
TorneoAmistoso
- valorBono: double
- bonoOtorgado: boolean
+ otorgarBono(ganador: Usuario): BonoDescubrimiento
Nota UML: los cupos reservados deben calcularse como ceil(0.2*cupoTotal) (redondeo hacia arriba).
bonos
TorneoCompetitivo
- tarifaEntrada: double
- pozoPremio: double
- porcentajePozoPremio: double
+ calcularPozo(): double
+ esElegiblePremioMetalico (u: Usuario): boolean
+ registrarPago (u: Usuario, monto: double): void
esElegiblePremioMetalico -> (false si empleado) registrarPago -> (aunque el pago sea tercerizado, guardas el registro)
torneo
0..*
bonoAplicado
0..1
InscripcionTorneo
- id_inscripcion: String
- fecha: Date
- cantidadCupos: int
- cuposReservadosUsados: int
- cuposReguladosUsados: int
- esEmpleado: boolean
- montoPagado: double
- pagoConfirmado: boolean
- elegiblePremioMetalico: boolean
cantidadCupos: int (1..3) cupos ReservadosUsados: int (0..cantidadCupos) cupos ReguladosUsados: int (0..cantidad Cupos) elegiblePremioMetalico -> (false si Empleado en Competitivo)
Restricción (nota) Para cada par (Usuario, Torneo) solo puede existir 0..1 inscripción. cantidadCupos <= 3.
BonoTorneoAmistoso
- codigo: String
- valor: double
- estado: String
- fechaotorgado: Date
- fechaUsado: Date (nullable)
nota: No cumulable-> venta no puede aplicar bono si ya aplico otros descuentos/ puntos
ganador